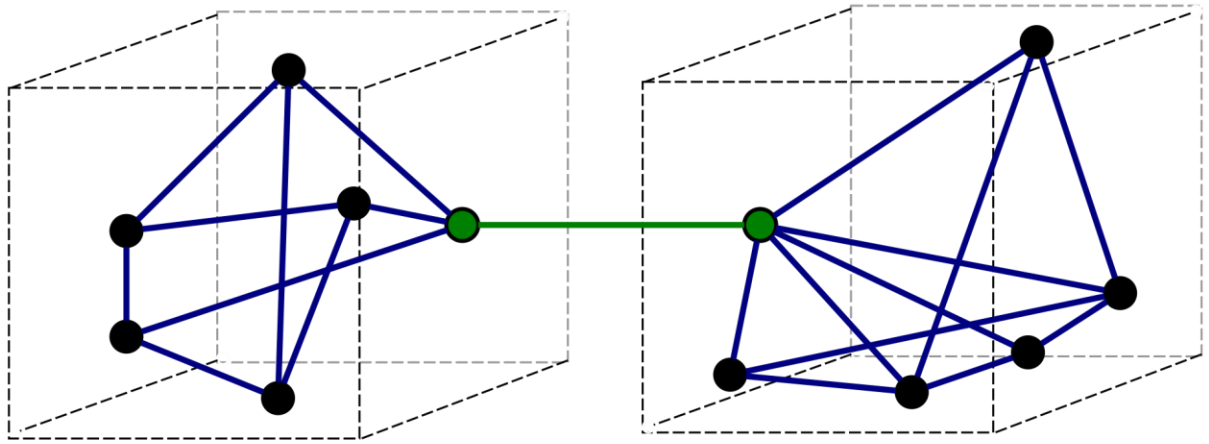
CS202: Software Tools and Techniques for CSE

Lecture 8

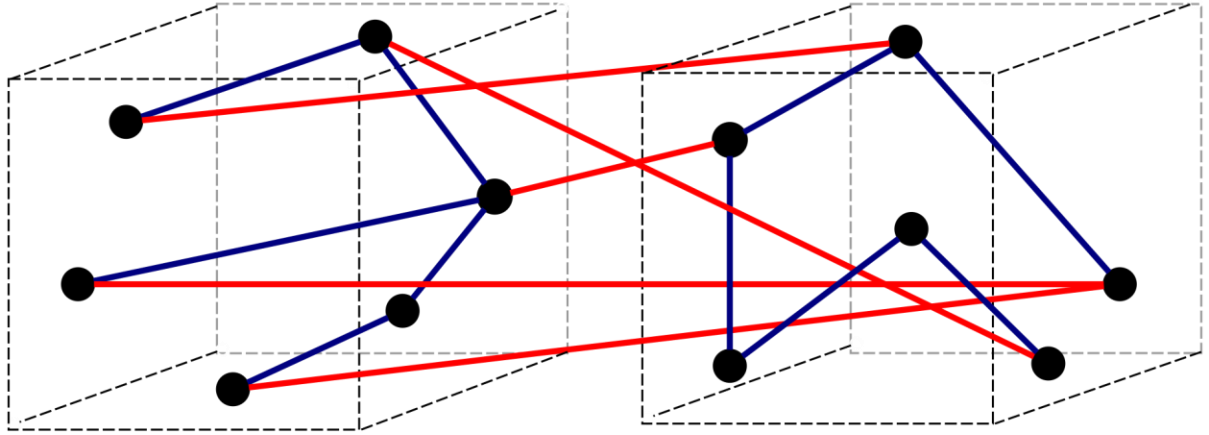
Shouvick Mondal

shouvick.mondal@iitgn.ac.in
January 2025

Coupling and Cohesion



a) Good (loose coupling, high cohesion)

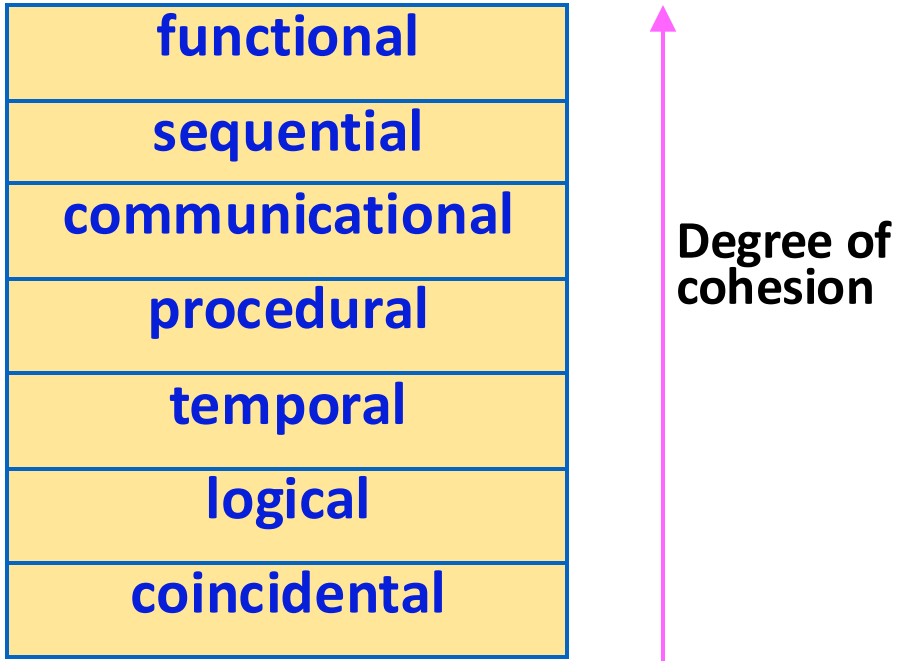


b) Bad (high coupling, low cohesion)

Classification of Cohesiveness

- Classification is often subjective:
  - Yet gives us some idea about cohesiveness of a module.
- By examining the type of cohesion exhibited by a module:
  - We can roughly tell whether it displays high cohesion or low cohesion.

Classification of Cohesiveness



Coincidental Cohesion

- The module performs a set of tasks:
  - Which relate to each other very loosely, if at all.
    - The module contains a random collection of functions.
    - Functions have been put in the module out of pure coincidence without any thought or design.

Coincidental Cohesion

```
def reverse_string(s):
    reversed_string = s[::-1]
    return reversed_string

def calculate_area(length, width):
    area = length * width
    return area

def send_email(recipient, subject, body):
    import smtplib
    # Code to connect to email server
    # and send email
```

```
def send_sms(number, message):
    # Code to send SMS

def generate_pdf(content):
    # Code to generate PDF

def encrypt_data(data, key):
    # Code to encrypt data
```

```
def format_date(date):
    # Code to format date

def log_message(message, level):
    # Code to log message

def generate_random_number(min, max):
    import random random_number = random.randint(min, max)
    return random_number
```

Logical Cohesion

- All elements of the module perform similar operations:
  - e.g. error handling, data input, data output, etc.
- An example of logical cohesion:
  - A set of print functions to generate an output report arranged into a single module.

Logical Cohesion

```
# All functions related to string manipulation
def reverse_string(s):
    reversed_string = s[::-1]
    return reversed_string

def capitalize_words(text):
    return " ".join(word.capitalize() for word in text.split())

def remove_punctuation(text):
    import string return "".join(c for c in text if c not in string.punctuation)
```

```
# Module for generating report output
def print_report_header(title):
    # Print report header and title

def print_section_header(section_name):
    # Print section header within report

def print_data_table(data, columns=None):
    # Print formatted table of data (optional columns)

def print_report_footer():
    # Print report footer and closing message
```

Temporal Cohesion

- The module contains tasks that are related by the fact:
  - All the tasks must be executed in the same time span.
- Example:
  - The set of functions responsible for
    - initialization,
    - start-up, shut-down of some process, etc.

Temporal Cohesion

```
# These functions all run during program startup
def init_database_connection():
    # Connect to database at program startup

def load_configuration_files():
    # Load configuration files before other modules

def register_event_listeners():
    # Set up event listeners for initial processes
```

```
# These functions execute when errors occur
def log_error_details(error):
    # Record error information for debugging

def notify_user_of_error(error):
    # Communicate the error to the user

def attempt_error_recovery(error):
    # Try to resolve the error if possible
```

Procedural Cohesion

- The set of functions of the module:
  - All part of a procedure (algorithm)
  - Certain sequence of steps have to be carried out in a certain order for achieving an objective,
    - e.g. the algorithm for decoding a message.

Procedural Cohesion

```
# This module provides functions for manipulating and processing images.
def resize_image(image_path, new_size):
    # Resize image to specified dimensions

def adjust_brightness(image, intensity):
    # Modify image brightness level

def apply_filter(image, filter_type):
    # Apply chosen filter effect to the image

def save_processed_image(image, output_path):
    # Save the processed image to a specified location
```

Communicational Cohesion

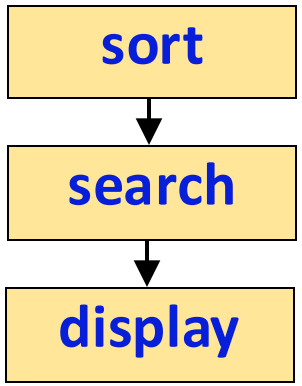
- All functions of the module:
  - Reference or update the same data structure,
- Example:
- The set of functions defined on an array or a stack.

Communicational Cohesion

```
class Stack: #The set of functions  defined on a stack
   def __init__(self):
      self.items = []
   def push(self, item):
      #Adds an item to the top of the stack.
      self.items.append(item)
   def pop(self):
      #Removes and returns the item from the top of the stack.
      if self.is_empty():
         raise Exception("Stack is empty")
      return self.items.pop()
   def is_empty(self):
      #Checks if the stack is empty.
      return len(self.items) == 0

# Example usage
my_stack = Stack()
my_stack.push(1)
print(my_stack.pop())
print(my_stack.is_empty())
```

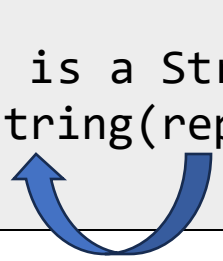
Sequential Cohesion

- Elements of a module form different parts of a sequence,
  - Output from one element of the sequence is input to the next.
  - Example:



Sequential Cohesion

```
def remove_whitespace(text):
   #Removes whitespace characters from a string.
   return "".join(text.split())
def convert_lowercase(text):
   #Converts all characters in a string to lowercase.
   return text.lower()
def replace_punctuation(text, replacement):
   #Replaces punctuation characters with a specified replacement.
   for char in "!@#$%^&*()-_+=|\\;:<>\"?/":
      text = text.replace(char, replacement)
   return text
def reverse_string(text):
   #Reverses the order of characters in a string.
   return text[::-1]

text = " This is a String! With Punctuation. "
t = reverse_string(replace_punctuation(convert_lowercase(remove_whitespace(text)), "-"))
print(t)
```



Functional Cohesion

- Different elements of a module cooperate:
  - To achieve a single function,
  - e.g. managing an employee's pay-roll.
- When a module displays functional cohesion,
  - We can describe the function using a single sentence.

Functional Cohesion

```
def play_song(song_path):
    #Loads and plays a specific song based on its path.
def pause_playback():
    #Pauses the currently playing song.
def resume_playback():
    #Resumes the playback of the paused song.
def adjust_volume(new_volume):
    #Changes the volume level of the playback.
def skip_to_next_song():
    #Skips to the next song in the playlist.
def skip_to_previous_song():
    #Skips to the previous song in the playlist.
def add_song_to_playlist(song_path):
    #Adds a song to the playlist (assuming playlist functionality exists).
def remove_song_from_playlist(song_path):
    #Removes a song from the playlist (assuming playlist functionality exists).
#Example usage
play_song("path/to/song.mp3")
#... other playback controls, playlist management
```

Determining Cohesiveness

- Write down a sentence to describe the function of the module
  - If the sentence is compound,
    - It has a sequential or communicational cohesion.
  - If it has words like “first”, “next”, “after”, “then”, etc.
    - It has sequential or temporal cohesion.
  - If it has words like initialize,
    - It probably has temporal cohesion.

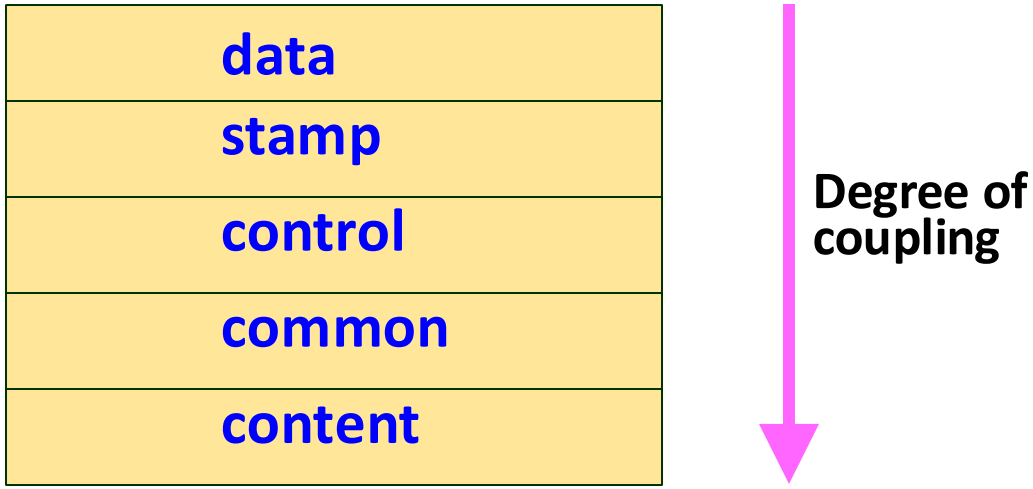
Coupling

- Coupling indicates:
  - How closely two modules interact or how interdependent they are.
  - The degree of coupling between two modules depends on their interface complexity.

Coupling

- There are no ways to precisely determine coupling between two modules:
  - Classification of different types of coupling will help us to approximately estimate the degree of coupling between two modules.
- Five types of coupling can exist between any two modules.

Classes of coupling



Data coupling

- Two modules are data coupled,
  - If they communicate via a parameter:
    - an elementary data item,
    - e.g an integer, a float, a character, etc.
  - The data item should be problem related:
    - Not used for control purpose.

Data coupling

```java
// Module B
public class ModuleB
{
    public static void performLogic(int data)
    {
        int processedData = ModuleA.processData(data);

        if (processedData > 10)
        {
            System.out.println("Result is > 10.");
        }
        else
        {
            System.out.println("Result is <= 10.");
        }
    }
}
```

```java
// Module A
public class ModuleA
{
    public static int processData(int data)
    {
        return data * 2;
    }
}
```

```java
// Main program
public class Main
{
    public static void main(String[] args)
    {
        int inputData = 5;
        ModuleB.performLogic(inputData);
    }
}
```

Stamp Coupling

- Two modules are stamp coupled,
  - If they communicate via a composite data item
    - such as a record in PASCAL
    - or a structure in C.

Stamp Coupling

```c
#include <stdio.h>

struct Employee
{ int empID; char name[50]; float salary; };

void display(struct Employee e)
{ printf("ID: %d, Name: %s, Salary: %.2f\n", e.empID, e.name, e.salary); }

void incrementSalary(struct Employee *e, float inc)
{ e->salary += inc; }

int main() {
    struct Employee emp = {101, "John Doe", 50000.0};
    display(emp); //read
    incrementSalary(&emp, 2000.0); //write
    display(emp); //read
    return 0;
}
```

Control Coupling

- Data from one module is used to direct:
  - Order of instruction execution in another.
- Example of control coupling:
  - A flag set in one module and tested in another module.

Control Coupling

```c
#include <stdio.h>
// Control-coupled modules
void processStep1(int *flag)
{
    // Set the flag
    *flag = 1;
}

void processStep2(int flag) {
    // Control flow based on the flag
    if (flag == 1)
    {
        printf("Processing Step 2 after Step 1.\n");
    } else {
        printf("Processing Step 2 without Step 1.\n");
    }
}
...
```

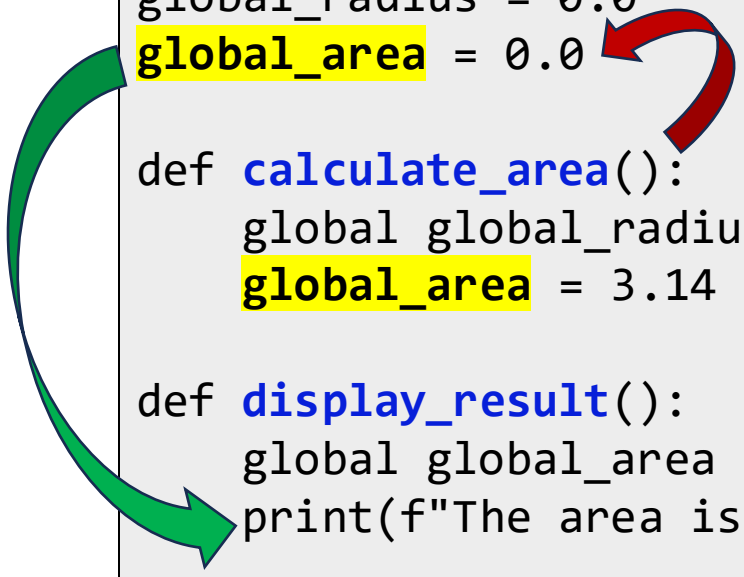
```c
...
int main()
{
    int flag = 0;

    processStep1(&flag);
    processStep2(flag);

    return 0;
}
```

Common Coupling

- Two modules are common coupled,
  - If they share some global data.

Common Coupling

```
# Comm-coupled modules with global variables
global_radius = 0.0
global_area = 0.0

def calculate_area():
    global global_radius, global_area
    global_area = 3.14 * global_radius ** 2

def display_result():
    global global_area
    print(f"The area is: {global_area}")

# Example usage
global_radius = 5.0
calculate_area()
display_result()
```



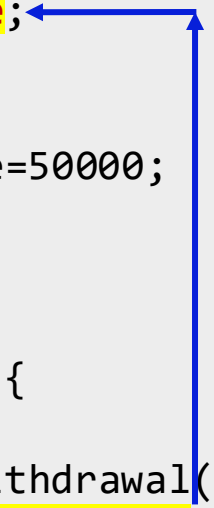
```
# Comm-coupled with global variables
global_counter = 0

def increment_counter():
    global global_counter
    global_counter += 1

def display_counter():
    global global_counter
    print(f"The counter: {global_counter}")

# Example usage
increment_counter()
display_counter()
```

Content Coupling

- Content coupling exists between two modules:
  - If they share code,
  - e.g, branching from one module into another module. This violates information hiding.
- The degree of coupling increases
  - from data coupling to content coupling.

Content Coupling

```
#include<iostream>
using namespace std;

class Account {
public:
    int balance;
    Account()
    {
        balance=50000;
    }
};

class Customer {
public:
    void makeWithdrawal(Account& account, int amount) {
        account.balance -= amount;
    }
    void print(Account& account)
    {
        cout<<account.balance;
    }
};
```



```
...
int main()
{
    Account a;
    Customer c;
    c.makeWithdrawal(a,1000);
    c.print(a);
    return(0);
}
```

Neat Hierarchy

- Control hierarchy represents:
  - Organization of modules.
  - Control hierarchy is also called program structure.
- Most common notation:
  - A tree-like diagram called structure chart.

Layered Design

- Essentially means:
  - Low fan-out
  - Control abstraction

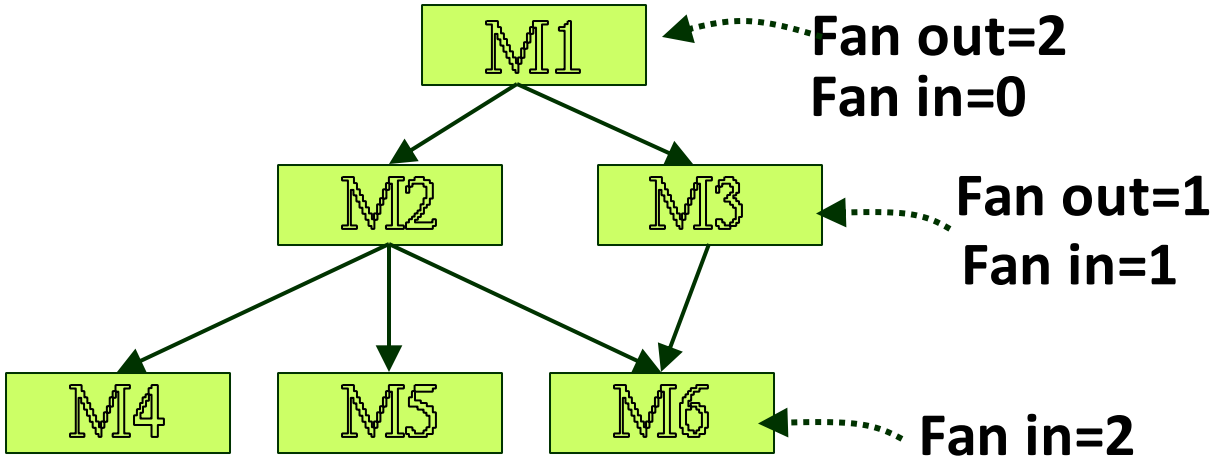
Characteristics of Module Hierarchy

- Depth:
  - Number of levels of control
- Width:
  - Overall span of control.
- Fan-out:
  - A measure of the number of modules directly controlled by given module.

Characteristics of Module Hierarchy

- Fan-in:
  - Indicates how many modules directly invoke a given module.
  - High fan-in represents code reuse and is in general encouraged.

Module Structure



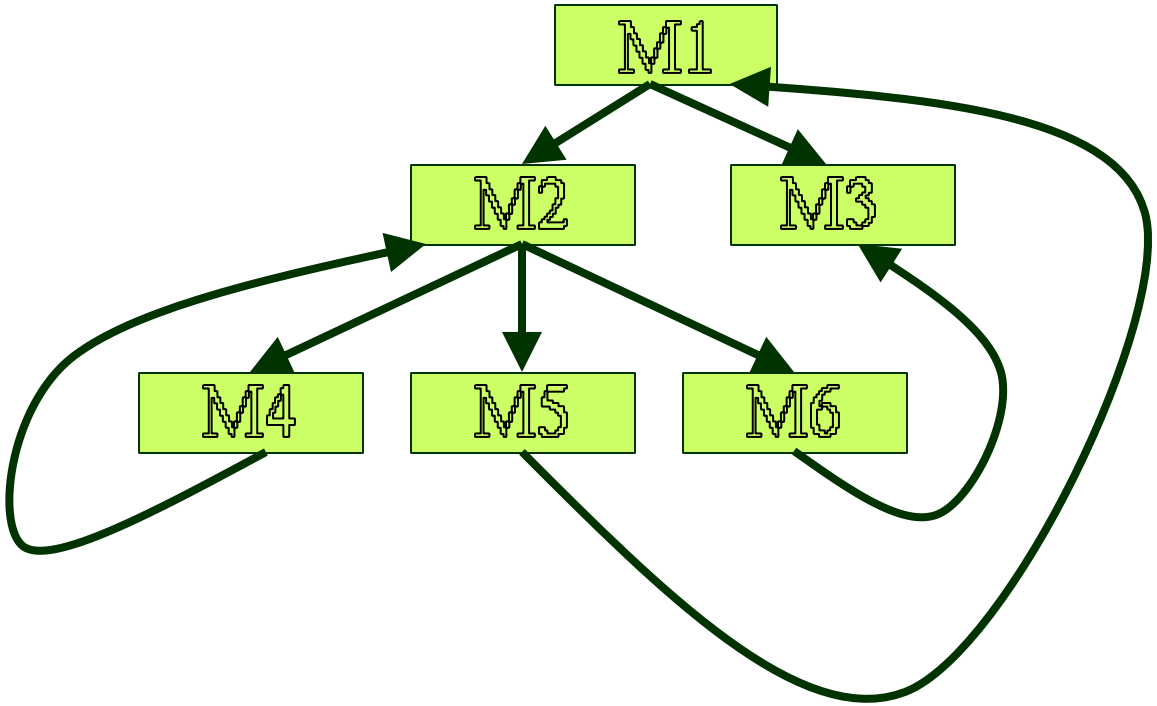
Layered Design

- A design having modules:
  - With high fan-out numbers is not a good design:
  - A module having high fan-out lacks cohesion.

Goodness of Design

- A module that invokes a large number of other modules:
  - Likely to implement several different functions:
  - Not likely to perform a single cohesive function.

Bad Design



Module Graphs

(structure charts)

`modulegraph` (https://pypi.org/project/modulegraph)

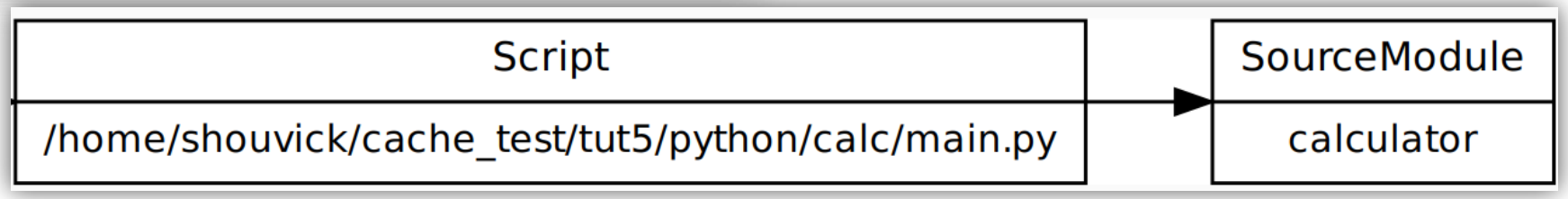
```
shouvick@shouvick:~/cache_test/tut5/python/calc$ modulegraph --help
usage: python3.10 -mmodulegraph [--help] [-d] [-q] [-m] [-x NAME] [-p PATH]
                                [-g] [-h]
                                SCRIPT [SCRIPT ...]

positional arguments:
  SCRIPT              scripts to analyse

options:
  --help              show this help message and exit
  -d                  Increase debug level
  -q                  Clear debug level
  -m, --modules       arguments are module names, not script files
  -x NAME             Add NAME to the excludes list
  -p PATH             Add PATH to the module search path
  -g, --dot           Output a .dot graph
  -h, --html          Output a HTML file
```

Module Graphs

(structure charts)

```python
# calculator.py

def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    if y != 0:
        return x / y
    else:
        return "Cannot divide by zero"
```

```python
# main.py
import calculator

# Using functions from the calculator module
result_add = calculator.add(5, 3)
result_subtract = calculator.subtract(8, 4)
result_multiply = calculator.multiply(2, 6)
result_divide = calculator.divide(10, 2)

# Displaying results
print(f"Addition: {result_add}")
print(f"Subtraction: {result_subtract}")
print(f"Multiplication: {result_multiply}")
print(f"Division: {result_divide}")
```



modulegraph (https://pypi.org/project/modulegraph)

Module Graphs

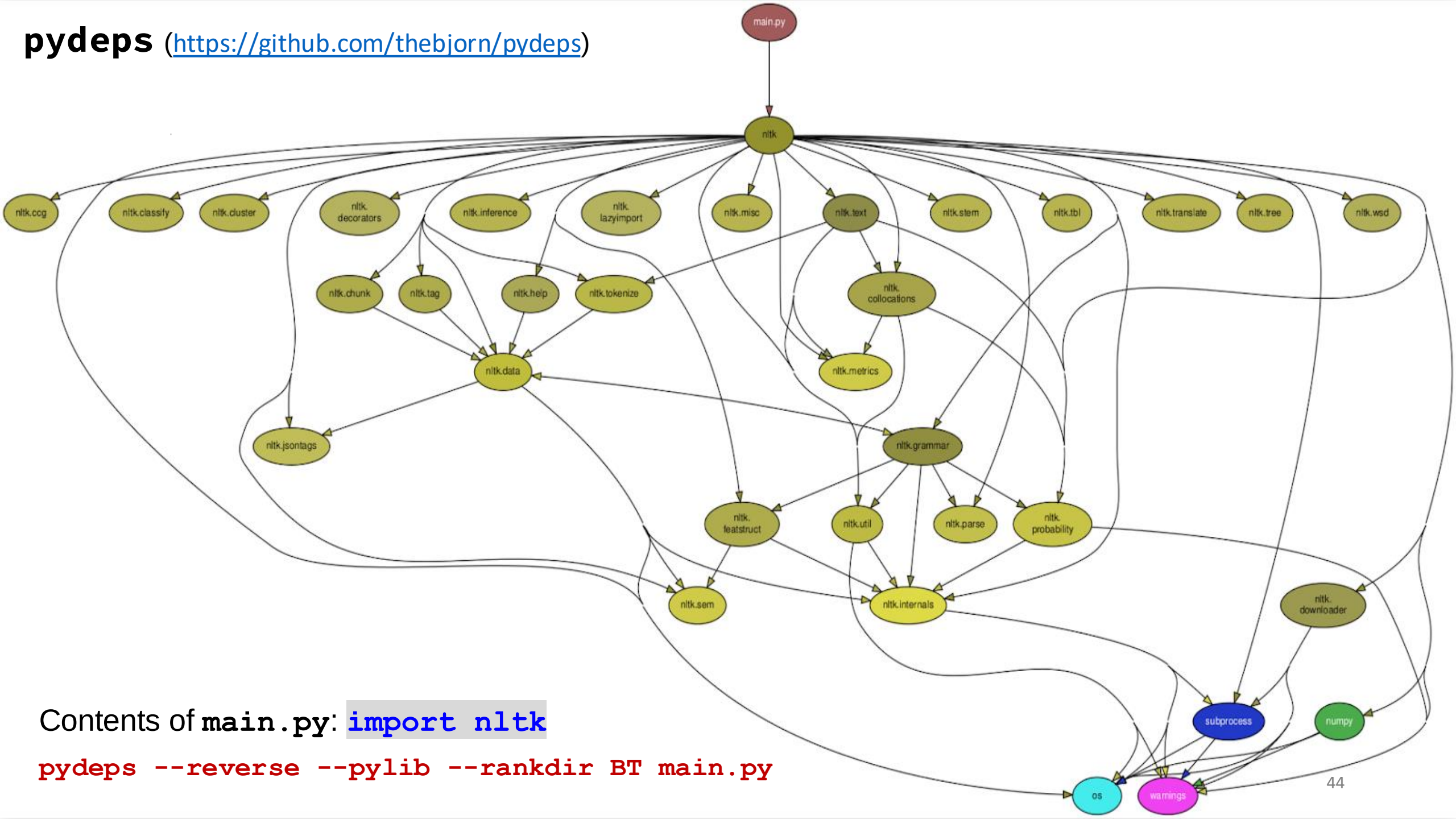
(structure charts)

`pydeps` (https://github.com/thebjorn/pydeps)

Contents of **`main.py`**: `import nltk`

```
pydeps --reverse --pylib --rankdir BT main.py
```

pydeps (https://github.com/thebjorn/pydeps)



Contents of **`main.py`**: **`import nltk`**

```
pydeps --reverse --pylib --rankdir BT main.py
```

LCOM{1..5} & YALCOM

LCOM.jar (java) (https://github.com/tushartushar/LCOM) for the LCOM{1..5}/ YALCOM metrics implemented/ proposed by Sharma & Spinellis (2020).

Do We Need Improved Code Quality Metrics?

The YALCOM metric

Sharma & Spinellis (2020)

```
Input: Type t
Output: LCOM value
G = initialize a graph
if isMetricComputable(t) then
    G.addVertex(t.methods())
    G.addVertex(t.attributes())
    G.addVertex(t.supertype().attributes())
    for m : t.methods() do
        G.addEdge(m, m.attributesAccessed())
        G.addEdge(m, m.methodInvocations())
    end
    d = G.disconnectedGraphs()
    if d > 1 then
        LCOM = d/t.methods().size()
    else
        LCOM = 0
    end
else
    LCOM = -1
end
```

```
isMetricComputable()
Input: t
Output: True/False
begin
    if t.methods().Count = 0 Or t.isInterface() then
        return False
    end
    return True
end
```

Java code

{m1}, {m2}, {m3}, {f1}, {f2}, {f3}

```
package lcom.testsubject;

public class AllDisconnected {
    public int f1, f2, f3;
    public void m1(){}
    public void m2(){}
    public void m3(){}
}
```

Output of **LCOM.jar**

LCOM1	LCOM2	LCOM3	LCOM4	LCOM5	YALCOM
3	0	6	6	1.5	1

LCOM4 essentially means the number of connected components

Java code

{m1} → {f1, f2, f3},
{m2} → {f2},
{m3} → {f3}

```
package lcom.testsubject;

public class Case1 {
    int f1, f2, f3;

    public void m1() {
        f1 = 0;
        f2 = 0;
        f3 = 0;
    }

    void m2() {
        f2 = 0;
    }

    void m3() {
        f3 = 0;
    }
}
```

Output of **`LCOM.jar`**

LCOM1	LCOM2	LCOM3	LCOM4	LCOM5	YALCOM
1	0	1	1	0.67	0

Java code

{m1} → {f1, f2},
{m2} → {f2},
{m3} → {f3}

```
package lcom.testsubject;

public class Case2 {
    int f1, f2;
    static int f3; //f3 is static:

    public void m1() {
        f1 = 0;
        f2 = 0;
        f3 = 0;
    }

    void m2() {
        f2 = 0;
    }

    void m3() {
        f3 = 0;
    }
}
```

Output of **LCOM.jar**

LCOM1	LCOM2	LCOM3	LCOM4	LCOM5	YALCOM
2	1	2	2	0.67	0

LCOM4 ignores static attributes of a class!

Java code

{m1} → {f1, f2},
{m2} → {f1, f2},
{m3} → {f3}

```
package lcom.testsubject;

public class Case3 {
    int f1, f2, f3;

    public void m1() {
        f1 = 0;
        f2 = 0;
    }

    void m2() {
        f1 = 0;
        f2 = 0;
    }

    void m3() {
        f3 = 0;
    }
}
```

Output of **LCOM.jar**

LCOM1	LCOM2	LCOM3	LCOM4	LCOM5	YALCOM
2	1	2	2	0.67	0.67

Java code	Output of **LCOM.jar**					
	LCOM1	LCOM2	LCOM3	**LCOM4**	LCOM5	**YALCOM**

`package lcom.testsubject;`
 	
`public class Case4Base {`
`  protected int f1;`
`}`
 	
{f1}
	0	0	1	1	0	-1

`package lcom.testsubject;`
 	
`public class Case4 extends Case4Base{`
`  int f2, f3;`
 	
`  public void m1() {`
`  f1 = 0;`
`  f2 = 0;`
`  f3 = 0;`
`  }`
 	
`  void m2() {`
`  f1 = 0;`
`  }`
 	
`  void m3() {`
`  f2 = 0;`
`  f3 = 0;`
`  }`
 	
`}`
 	
{m1} → {f1, f2, f3},
{m2} → {f1},
{m3} → {f2, f3}
	2	1	2	2	0.5	0

Java code

{m1} → {f1},
{m2} → {f2},
{m3} → {f3}

```
package lcom.testsubject;

public class Case5 {
    int f1, f2, f3;

    public void m1() {
        f1 = 0;
    }

    void m2() {
        f2 = 0;
    }

    void m3() {
        f3 = 0;
    }
}
```

Output of **LCOM.jar**

LCOM1	LCOM2	LCOM3	LCOM4	LCOM5	YALCOM
3	3	3	3	1	1

Java code

{m1} → {f1, f2, f3},
{m2} → {f2, m3},
{m3}

```
package lcom.testsubject;

public class Case6 {
    int f1, f2, f3;

    public void m1() {
        f1 = 0;
        f2 = 0;
        f3 = 0;
    }

    void m2() {
        f2 = 0;
        m3();
    }

    void m3() {
    }
}
```

Output of **`LCOM.jar`**

LCOM1	LCOM2	LCOM3	LCOM4	LCOM5	YALCOM
2	1	2	1	0.83	0

Java code {m1}, {m2}, {m3}	Output of **LCOM.jar**					
	LCOM1	LCOM2	LCOM3	**LCOM4**	LCOM5	**YALCOM**

`package lcom.testsubject;`
 	
`public interface Case7 {`
`  int m1();`
`  int m2();`
`  int m3();`
`}`
	3	0	3	3	0	-1

Java code

{m1},
{m2} → {m3},
{m3}

```
package lcom.testsubject;

public class Case8 {
    public void m1() {
    }

    void m2() {
        m3();
    }

    void m3() {
    }
}
```

Output of **`LCOM.jar`**

LCOM1	LCOM2	LCOM3	LCOM4	LCOM5	YALCOM
3	0	3	2	0	0.67

Java code {f1}, {f2}, {f3}	Output of **LCOM.jar**					
	LCOM1	LCOM2	LCOM3	**LCOM4**	LCOM5	**YALCOM**

`package lcom.testsubject;`
 	
`public class Case9 {`
`  public int f1, f2, f3;`
`}`
	0	0	3	3	0	-1

Java code

{main}

```
package lcom.testsubject;

public class Main {

    public static void main(String[] args) {
        //Just a dummy application
        //Look at the different cases where w
    }
}
```

Output of **LCOM.jar**

LCOM1	LCOM2	LCOM3	LCOM4	LCOM5	YALCOM
0	0	1	1	0	0

Texts, References, and Acknowledgements

Textbook:

- Sharp, J. (2022). *Microsoft Visual C# Step by Step*, 10th edition, Microsoft Press.
- Watson, K., Nagel, C., Pedersen, J. H., Reid, J. D., & Skinner, M. (2008). *Beginning Microsoft Visual C# 2008*. John Wiley & Sons.
- Mark J. Price (2024). *C# 13 and .NET 9 – Modern Cross-Platform Development Fundamentals*, 9th edition, Packt Publishing Ltd.

Reference:

- Rajib Mall. 2014. Fundamentals of Software Engineering (4th. edition). Prentice-Hall of India Pvt.Ltd.
- Soni, M. (2016). *DevOps for Web Development*. Packt Publishing Ltd.
- Yusuf Sulistyo Nugroho, Hideaki Hata, and Kenichi Matsumoto. 2020. *How different are different diff algorithms in Git? Use* `--histogram` *for code changes*. Empirical Softw. Engg. 25, 1 (Jan 2020), 790–823.